BTK AKADEMİ

İLERİ SEVİYE PYTHON PROGRAMLAMA DİLİ

Decorator Fonksiyonlar

EBRU AYDOĞAN

Decorator Fonksiyonlar

- Decorator Nedir ve Ne İşe Yarar?
  - Bir fonksiyonun koduna doğrudan dokunmadan, ona yeni özellikler eklememizi sağlayan "sarmalayıcı" (wrapper) yapılardır.
  - **Kod Tekrarını Önler:** Birçok fonksiyonda ortak olan işlemleri (Örn: Log tutma, yetki kontrolü, süre ölçme) tek bir yerde toplar.
  - **Temiz Kod (Clean Code):** Fonksiyonun ana işini, yardımcı işlerden (güvenlik, hata yönetimi vb.) ayırır.
  - **Esneklik:** İstediğiniz fonksiyona @decorator_adi yazarak anında yeni bir yetenek kazandırabilirsiniz.

Decorator Fonksiyonlar

- Neden Decorator Kullanırız?
  - **Senaryo:** Elimizde 50 tane farklı fonksiyon olduğunu düşünün. Hepsinin başında "Kullanıcı giriş yaptı mı?" veya "Bu fonksiyonu çalıştıranın yetkisi var mı?" diye kontrol etmek istiyoruz.
  - **Klasik Yol:** Her birinin içine if yetkili_mi(): yazmak. (Çok zahmetli!)
  - **Decorator Yolu:** Tek bir "Kontrolcü" fonksiyon yazıp, istediğimiz fonksiyonun üzerine @kontrolcu yazmak.
  - Decorator, ana fonksiyonun koduna dokunmadan ona yeni bir yetenek/davranış ekleyen bir "sarmalayıcıdır".

Decorator Fonksiyonlar

- Decorator Nasıl Çalışır?
  - **Hediye Kutusu:** Ana fonksiyonun bir "hediye" olsun. Decorator ise o hediyenin etrafına sarılan "hediye paketi"dir.
    - Hediye paketini açarsın (Ön işlemler).
    - Hediye kutusunu görürsün (Asıl fonksiyon).
    - Hediye paketini kapatırsın (Son işlemler).

BTK AKADEMİ

Decorator Fonksiyonlar

- Örnek: Basit Bir Decorator

```
def susleyici(func):
    def wrapper():
        print("--- Paket Açıldı (Ön İşlem) ---")
        func()  # Hediyeyi burada açtık
        print("--- Paket Kapandı (Son İşlem) ---")
    return wrapper

@susleyici
def selamla():
    print("Merhaba Dünya!")

selamla()
```

BTK AKADEMİ

Parametre Alan Fonksiyonlar İçin Çözüm

- Eğer fonksiyonumuz dışarıdan bilgi alıyorsa (topla(a,b) gibi), decorator'ımız da bu bilgiyi alıp sarmalanan fonksiyona "aktarmalıdır".
- Çözüm: *args ve **kwargs kullanımı.

BTK AKADEMİ

Parametre Alan Fonksiyonlar İçin Çözüm

```python
def rapor_decorator(func):
    def wrapper(*args, **kwargs):
        # Parametreleri al, fonksiyona aktar
        print(f"-> {func.__name__} çağrıldı.")
        sonuc = func(*args, **kwargs)
        print("-> İşlem başarıyla bitti.")
        return sonuc
    return wrapper

@rapor_decorator
def topla(a, b):
    return a + b
```

BTK AKADEMİ

Decorator'ların Altın Kuralları

- **İç İçe Fonksiyon:** Dışta bir fonksiyon, içinde wrapper fonksiyonu olmalı.
- **Fonksiyonu Geri Döndür:** return wrapper satırı mutlaka olmalı.
- **Parametreleri Unutma:** *args, **kwargs ile fonksiyonun tüm parametrelerini yakalamalıyız.
- **Sonucu Dön:** return func(...) ile ana fonksiyonun sonucunu mutlaka döndürmeliyiz.

BTK AKADEMİ

Decorator'lara Argüman Gönderme

- Neden Argümana İhtiyaç Duyarız?
  - Sorun: Sabit bir decorator her zaman aynı şeyi yapar. Ancak farklı durumlar için farklı kurallarımız olabilir.
  - Örnek: Bir yetki_kontrol decorator'ımız olsun. Bazen "admin" olanları, bazen sadece "user" olanları geçirmek isteyelim.
  - Çözüm: Decorator'ı, decorator üreten bir "fabrika" fonksiyonuna dönüştürüyoruz.

Decorator'lara Argüman Gönderme

- Örnek – Yetki Kontrolü

```
def yetki_kontrol(gereken_rol): # 1. Katman: Fabrika
    def decorator(func):        # 2. Katman: Decorator
        def wrapper(*args, **kwargs): # 3. Katman: Wrapper
            kullanici_rolu = "user" # Gerçek projede bunu dışarıdan alacağız

            if kullanici_rolu == gereken_rol:
                return func(*args, **kwargs)
            else:
                return f"Hata: Bu işlem için {gereken_rol} olmalısınız!"
        return wrapper
    return decorator
```

BTK AKADEMİ

Decorator'lara Argüman Gönderme

- Örnek - Yetki Kontrolü

```
# Kullanım:
@yetki_kontrol("admin")
def admin_sayfasi():
    return "Admin paneline hoş geldiniz."

print(admin_sayfasi())
# Çıktı: Hata: Bu işlem için admin olmalısınız!
```

BTK AKADEMİ

Decorator'lara Argüman Gönderme

- Mantık Özeti
  - Eğer decorator'a parametre gönderiyorsak, 3 tane def iç içe olmak zorundadır.
  - gereken_rol değişkeni, wrapper fonksiyonu içinde "hatırlanır" (Closure mantığı sayesinde).
  - Bu sayede, kodu değiştirmeden sadece parametreyi değiştirerek farklı yetki seviyeleri oluşturabiliriz.

Örnek

- **Senaryo:** Bir fonksiyonun kaç kere çalışmasını istiyorsak, ona dışarıdan parametre olarak vereceğiz. Örneğin @tekrarla(3) yazarsak fonksiyon 3 kez çalışacak.

```
def tekrarla(n): # 1. Katman: Fabrika (Dışarıdan sayıyı alıyor)
    def decorator(func): # 2. Katman: Süsleyici
        def wrapper(*args, **kwargs): # 3. Katman: Asıl işi yapan paket
            for i in range(n):
                print(f"--- Çalışma sayısı: {i+1} ---")
                sonuc = func(*args, **kwargs)
            return sonuc
        return wrapper
    return decorator
```

BTK AKADEMİ

Örnek

- **Senaryo:** Bir fonksiyonun kaç kere çalışmasını istiyorsak, ona dışarıdan parametre olarak vereceğiz. Örneğin @tekrarla(3) yazarsak fonksiyon 3 kez çalışacak.

```
# Kullanım:
@tekrarla(3)
def merhaba_de(isim):
    print(f"Merhaba {isim}!")

merhaba_de("BTK Akademi")
```

BTK AKADEMİ

Uygulama 1: Performans İzleyicisi

- **Görev:** Bir fonksiyonun çalışması için geçen süreyi hesaplayan timer_decorator adında bir decorator yazın.
- **İstekler:**
  - Decorator, sarmaladığı fonksiyonun çalışması bittiğinde, ekrana "[Fonksiyon Adı] fonksiyonu [Süre] saniyede çalıştı." mesajını yazdırmalıdır.
  - time kütüphanesini kullanmalısınız.
  - Decorator, aldığı fonksiyon kaç parametre alırsa alsın çalışmalıdır (*args, **kwargs kullanımı zorunludur).

BTK AKADEMİ

Uygulama 2: Güvenlik Bekçisi

- **Görev:** @auth_decorator("admin") şeklinde kullanıldığında, sadece rolü "admin" olanların fonksiyonu çalıştırabilmesini sağlayan bir decorator fabrikası yazın.
- **İstekler:**
  - Decorator, parametre olarak gereken_rol bilgisini almalıdır.
  - Sistemde tanımlı bir current_user = {"role": "user"} sözlüğü varmış gibi düşünün.
  - Eğer kullanıcının rolü, @ ile verilen rol ile eşleşmiyorsa, fonksiyonu çalıştırmayın ve "Erişim Reddedildi: Yetersiz Yetki" hatasını döndürün.
  - Eşleşiyorsa, fonksiyonu normal şekilde çalıştırıp sonucunu döndürün.